Interior-Point Methods for Constrained Quadratic Programming: A Theoretical and Computational Study

Marco Tamberi - Vincenzo Rocchi

March 11, 2026

Contents

1 Introduction **3**
1.1 Problem Formulation . . . 3

2 A Brief Preamble to Interior-Point Methods **4**
2.1 Dual Methods . . . 4
2.2 Barrier Methods . . . 4

3 Interior-Point Methods **6**
3.1 KKT Conditions and Relaxation . . . 6
3.2 Newton Step and Computational Aspects . . . 6

4 Practical Relevance and Applications **8**

5 Pseudocode **9**

6 From Pseudocode to Python **10**
6.1 The H Formulation: Derivation and Correctness . . . 10
6.2 Implementation Structure . . . 10
6.3 Implementation Notes . . . 10

7 Experimental Setup and Results **12**
7.1 Setup . . . 12
7.2 Suite summary . . . 12
7.3 Detailed single-instance comparison . . . 13

8 Conclusions **14**

A Configuration Parameters **15**

1 Introduction

This report analyzes the use of interior-point methods (IPMs) for solving constrained convex quadratic programs (QPs) with polyhedral feasible regions. The structure is simple: first, we provide a clear theoretical motivation for why applying IPMs to this specific class of problems is the most suitable choice. Then, we move to a practical perspective, evaluating their performance on representative instances and comparing the results with those obtained using a general-purpose QP solver.

Constrained QPs arise in numerous application areas. In finance, portfolio models balance expected return and risk under budget and regulatory constraints. In agriculture and operations research, planning models allocate limited resources to maximize yields or profits. In machine learning, support vector machines (SVMs) are trained by solving a convex QP with linear constraints, where the separating hyperplane corresponds to the solution of such a program.

In our specific setting, **Project 34**, the feasible region imposes *simplex* constraints blockwise. This situation is common whenever decision variables encode probabilities, convex weights, or categorical assignments. The structure naturally suggests the use of IPMs, and in particular motivates a closer look at primal–dual feasible-start variants.

1.1 Problem Formulation

We formalize the optimization problem at the core of the project. Consider the quadratic program **(P)** defined over a set K of disjoint simplices:

$$\min \left\{ \tfrac{1}{2}x^\top Qx + q^\top x \ : \ \sum_{i \in I^k} x_i = 1 \ \ \forall\, k \in K, \ \ x \geq 0 \right\},$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of $\{1, \ldots, n\}$, and $Q \in \mathbb{R}^{n \times n}$ is symmetric positive semidefinite.

Notation. We adopt the standard convention for the quadratic objective

$$\tfrac{1}{2}x^\top Qx + q^\top x,$$

where the factor $\frac{1}{2}$ simplifies derivatives: for symmetric Q, $\nabla(\frac{1}{2}x^\top Qx) = Qx$ and $\nabla^2(\frac{1}{2}x^\top Qx) = Q$.

The objective consists of a quadratic form $\frac{1}{2}x^\top Qx$ and a linear term $q^\top x$, with $q \in \mathbb{R}^n$. Since Q is positive semidefinite, the quadratic form is convex, which allows us to state that (P) is a convex quadratic program.

More precisely, the curvature of the quadratic form depends on the eigenvalues of Q: larger eigenvalues correspond to steeper curvature in the associated eigen-directions. If $Q \succ 0$, the function is strictly convex and has a unique unconstrained minimum, while if $Q \succeq 0$ but singular, the function may be flat along some directions, but still remains convex.

When the linear term $q^\top x$ is included, the analysis extends as follows. If $Q \succ 0$, the problem remains strictly convex and admits a unique minimum. If $Q \succeq 0$, a finite minimum exists if and only if $q \in \mathrm{Im}(Q)$, equivalently $q \perp \ker(Q)$. In this case, the quadratic term alone attains its minimum value 0 on $\ker(Q)$, but the addition of the linear term may destroy boundedness unless q is flat along $\ker(Q)$.

The domain is defined by linear equalities and nonnegativity constraints. The indices are divided into disjoint subsets I^k, each representing a block of variables subject to

$$\sum_{i \in I^k} x_i = 1, \qquad x_i \geq 0.$$

In other words, each block of variables lies on a simplex.

Geometrically, each block I^k defines a standard simplex. Since the blocks are disjoint, the overall domain is the Cartesian product of these simplices:

$$\mathcal{X} \ = \ \prod_{k \in K} \Delta^{|I^k|-1},$$

where $\Delta^{|I^k|-1}$ denotes the standard simplex of dimension $|I^k| - 1$.

The feasible region is thus a closed, bounded (and therefore compact) convex polytope. Interior points correspond to strictly positive vectors ($x_i > 0$), while vertices represent "pure" choices in which, for each block, a single coordinate equals 1 and the others are 0.

2 A Brief Preamble to Interior-Point Methods

Before introducing interior-point methods, we need to provide some theoretical background to better understand the context and how these methods work. A standard convex quadratic program can be written as

$$\min \left\{ \tfrac{1}{2}x^\top Qx + q^\top x \ : \ Ax \le b \right\},$$

with $Q \succeq 0$. This is referred to as the *primal* problem. In many cases, analysis is simplified by considering the associated *dual* problem.

Consistency with our model. In our specific instance (product of simplices) the constraints take the form $Ex = \mathbf{1}$ and $x \ge 0$; we will switch to this equalities+nonnegativity form when writing the KKT system for our algorithm.

2.1 Dual Methods

The starting point is the Lagrangian function:

$$L(x, \lambda) \ = \ \tfrac{1}{2}x^\top Qx + q^\top x \ + \ \lambda^\top (Ax - b), \qquad \lambda \ge 0,$$

where the multipliers $\lambda \in \mathbb{R}^m$ penalize constraint violations. The associated dual function is

$$\psi(\lambda) \ = \ \inf_{x \in \mathbb{R}^n} L(x, \lambda),$$

which provides a lower bound to the optimal value $\nu(P)$ of the primal problem. The dual problem is therefore

$$(D) \qquad \max_{\lambda \ge 0} \ \psi(\lambda).$$

The dual perspective is powerful: under convexity assumptions and a constraint qualification such as Slater's condition (existence of a strictly feasible point), *strong duality* holds:

$$\nu(D) \ = \ \nu(P),$$

so the optimal value of the dual coincides with that of the primal. This enables constrained problems to be solved indirectly through their dual, which may in some cases be simpler or better structured.

However, the dual approach also presents some limitations. The dual function $\psi(\lambda)$ is concave but may be nonsmooth (not even C^1 if the primal objective is not strictly convex), and the primal vector $x(\lambda)$ is typically infeasible until the end of the iterations. In other words, dual methods provide valid lower bounds early, but recovering a feasible primal solution is less immediate.

Remark (specialization to our constraints). For later use, when $Ex = \mathbf{1}$ and $x \ge 0$, a convenient Lagrangian is

$$L(x, y, z) \ = \ \tfrac{1}{2}x^\top Qx + q^\top x \ + \ y^\top (Ex - \mathbf{1}) \ - \ z^\top x, \qquad z \ge 0,$$

with y free (equalities) and z the multipliers for nonnegativity.

Algorithmic Desiderata. These limitations motivate methods that:

1. yield an unconstrained dual optimization problem,
2. operate with a smooth (preferably C^2) dual function ψ,
3. maintain (strict) primal feasibility throughout the iterations.

2.2 Barrier Methods

To address the limitations of dual methods, we now look at *barrier methods*. These replace explicit inequality constraints with penalization terms that diverge as iterates approach the boundary of the feasible region. A natural choice is the *logarithmic barrier*:

$$f_\gamma(x) \ = \ f(x) \ - \ \gamma \sum_{i=1}^{m} \ln\big(b_i - A_i x\big), \qquad \gamma > 0,$$

where γ controls the strength of the penalization.

This transformation ensures that $f_\gamma(x)$ is infinite for infeasible points ($x \notin X$) or boundary points ($x \in \partial X$), so all iterates remain strictly feasible. Moreover, on the interior of X, f_γ is C^2 and, if f is convex, then f_γ is strictly convex, enabling an effective use of Newton's method.

A key concept is the *central path*: as the barrier parameter γ varies, we obtain a family of problems (P_γ) with solutions x_γ that converge to the optimum $x^\star$ of the original problem. In particular,

$$\gamma \to \infty \;\Rightarrow\; x_\gamma \to \text{analytic center of } X, \qquad \gamma \to 0 \;\Rightarrow\; x_\gamma \to x^\star.$$

This continuous trajectory, the central path, serves as a stable guide for the iterates.

From a computational perspective, barrier methods have notable advantages. Newton's method applied to f_γ converges very rapidly when initialized near the central path, often achieving superlinear or even quadratic convergence.

Iteration complexity. Classical analyses give an iteration bound of

$$O\big(m \log(1/\varepsilon)\big),$$

where m is the number of inequality constraints. Using the self-concordant barrier framework, this improves to

$$O\big(\sqrt{\nu}\, \log(1/\varepsilon)\big),$$

where ν is the barrier parameter (equal to the number of barrier terms in the logarithmic case). The dominant cost per iteration lies in solving the Newton linear systems.

In the generic form $Ax \le b$, we have $\nu = m$. In our formulation $Ex = \mathbf{1}$, $x \ge 0$, the barrier applies only to the n nonnegativity constraints, so $\nu = n$. For dense problems, solving the Newton systems typically costs at least $O(n^3)$ per iteration.

Remark (link to our constraints). For $Ex = \mathbf{1}$ and $x > 0$, the barrier term would specialize to $-\gamma \sum_{i=1}^n \ln x_i$, while the equalities are handled by Lagrange multipliers. Theoretically, on the central path the barrier weight γ and the complementarity parameter μ coincide; however, the method we adopt (Section 3) works directly with the primal–dual KKT relaxation and never forms the barrier function explicitly.

3 Interior-Point Methods

Barrier methods provide the conceptual foundation—transforming a constrained problem into a smooth one on the interior of the feasible set—but in practice we do not minimize the barrier function directly. Instead, the most effective approach is the *primal–dual interior-point method*, which works with the Karush–Kuhn–Tucker (KKT) optimality conditions and relaxes the complementarity requirement $x_i z_i = 0$ to $x_i z_i = \mu$, driving $\mu \to 0$. This formulation solves the primal and dual problems simultaneously, keeping both nearly consistent throughout the iterations, and is widely used for Linear Programming (LP), Quadratic Programming (QP), and Semidefinite Programming (SDP) problems.

3.1 KKT Conditions and Relaxation

Consider the constrained quadratic program

$$(P) \qquad \min \ \tfrac{1}{2}x^\top Qx + q^\top x \quad \text{s.t.} \quad Ax \le b,$$

which can be reformulated by introducing slack variables $s \ge 0$ (a standard trick to turn inequalities into equalities) as

$$Ax + s = b, \qquad Qx + A^\top \lambda + q = 0, \qquad \lambda \ge 0,\ s \ge 0, \qquad \lambda_i s_i = 0.$$

These are precisely the Karush–Kuhn–Tucker (KKT) conditions:

1. primal feasibility (KKT-F): $Ax + s = b$, $s \ge 0$;
2. dual feasibility (KKT-G): $Qx + A^\top \lambda + q = 0$, $\lambda \ge 0$;
3. complementarity (KKT-CS): $\lambda_i s_i = 0$ for $i = 1, \dots, m$.

The primal–dual interior-point method replaces the complementarity condition $\lambda_i s_i = 0$ with a relaxed version:

$$\lambda_i s_i \ = \ \mu, \qquad i = 1, \dots, m,$$

with $\mu > 0$ gradually decreased to zero. Rather than enforcing complementarity exactly (which is numerically challenging), this approach follows a *central path* that connects the solutions of the perturbed problems (P_μ) to the optimal solution of the original problem. For the logarithmic barrier, along the central path, with weight γ, the relation is $\mu = \gamma$.

3.2 Newton Step and Computational Aspects

At the heart of primal–dual interior-point methods is the Newton step applied to the relaxed KKT system. By linearizing the nonlinear complementarity conditions, we obtain a linear system whose solution gives the search direction for the next iterate. The main computational effort in each iteration is solving this system, which usually requires heavy matrix factorizations.

Explicitly, introducing $\Lambda = \operatorname{diag}(\lambda)$ and $S = \operatorname{diag}(s)$, the complementarity condition is written as $\Lambda S u = \mu u$. Applying Newton's method gives the block linear system

$$\begin{bmatrix} Q & A^\top & 0 \\ A & 0 & I \\ 0 & S & \Lambda \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \\ \Delta s \end{bmatrix} = - \begin{bmatrix} r_D \\ r_P \\ r_C \end{bmatrix},$$

with residuals $r_D = Qx + A^\top \lambda + q$, $r_P = Ax + s - b$, and $r_C = \Lambda S u - \mu u$. For dense problems this entails a worst-case cost of $O(n^3)$ per iteration. To improve efficiency and stability, predictor–corrector schemes are often used: an affine-scaling predictor provides a descent direction, then a correction step reintroduces the centering term to follow the central path more closely. This reduces the number of iterations and accelerates convergence in practice.

Specialization to our constraints ($Ex = \mathbf{1}$, $x \ge 0$). We now apply the general framework to our setting. Recalling the Lagrangian introduced in Section 2.1,

$$L(x, y, z) = \tfrac{1}{2}x^\top Qx + q^\top x + y^\top (Ex - \mathbf{1}) - z^\top x, \qquad z \ge 0,$$

the KKT conditions become

$$\begin{aligned}
&\text{primal feasibility:} && Ex = \mathbf{1}, \quad x \geq 0,\\
&\text{dual feasibility:} && Qx + q + E^\top y - z = 0, \quad z \geq 0,\\
&\text{complementarity:} && x_i z_i = 0, \quad i = 1, \ldots, n.
\end{aligned}$$

Here z plays the role of the dual multipliers for $x \geq 0$ (analogous to λ for inequalities in the generic case). Complementarity is relaxed to $x_i z_i = \mu$ with $\mu > 0$ decreased to zero. Let $X = \operatorname{diag}(x)$ and $Z = \operatorname{diag}(z)$, and define residuals

$$r_D = Qx + q + E^\top y - z, \qquad r_P = Ex - \mathbf{1}, \qquad r_C = XZ\mathbf{1} - \mu\mathbf{1}.$$

Linearizing the KKT system yields the Newton equations

$$\begin{bmatrix} Q & E^\top & -I \\ E & 0 & 0 \\ Z & 0 & X \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta z \end{bmatrix} = - \begin{bmatrix} r_D \\ r_P \\ r_C \end{bmatrix}.$$

Elimination and reduced system. The Newton equations can be simplified through block elimination, yielding a Schur-complement system in $(\Delta x, \Delta y)$ which makes it more efficient to solve in practice.

For a **feasible-start** primal–dual interior-point method we assume that the iterates always satisfy primal and dual feasibility, $Ex = \mathbf{1}$ and $Qx + E^\top y - z + q = 0$, so that only the complementarity condition is relaxed. From the third block row of the Newton system we obtain

$$X\,\Delta z = -\,r_C - Z\,\Delta x, \qquad r_C = XZ\mathbf{1} - \mu\mathbf{1}.$$

Hence

$$\Delta z = -\,X^{-1} r_C - X^{-1} Z\,\Delta x.$$

Substituting into the first block row $Q\Delta x + E^\top \Delta y - \Delta z = -\,r_D$ gives the reduced (Schur–complement) system in $(\Delta x, \Delta y)$:

$$\boxed{(Q + X^{-1}Z)\,\Delta x + E^\top \Delta y = -\,r_D - X^{-1} r_C}, \qquad E\,\Delta x = -\,r_P.$$

Because a feasible-start method keeps both primal and dual constraints exactly satisfied at each iteration, the residuals r_P and r_D vanish identically, and only the complementarity residual r_C remains to be corrected. Therefore, the linearized system simplifies further to

$$\boxed{(Q + X^{-1}Z)\,\Delta x + E^\top \Delta y = -\,X^{-1} r_C, \qquad E\,\Delta x = \mathbf{0}.}$$

This form corrects only complementarity errors while maintaining primal and dual feasibility at every iteration.

Step sizes and update. Choose primal and dual step lengths by the fraction-to-the-boundary rule:

$$\alpha_{\text{pri}} = \min\Big(1,\ 0.99 \cdot \min_{i:\,\Delta x_i < 0} -\tfrac{x_i}{\Delta x_i}\Big), \qquad \alpha_{\text{dual}} = \min\Big(1,\ 0.99 \cdot \min_{i:\,\Delta z_i < 0} -\tfrac{z_i}{\Delta z_i}\Big),$$

and set $\alpha = \min(\alpha_{\text{pri}}, \alpha_{\text{dual}})$ to ensure $x + \alpha\Delta x > 0$ and $z + \alpha\Delta z > 0$.

Variable updates.

$$x \leftarrow x + \alpha\Delta x, \qquad y \leftarrow y + \alpha\Delta y, \qquad z \leftarrow z + \alpha\Delta z.$$

Complementarity update. After each iteration, update the complementarity measure as

$$\mu = \frac{x^\top z}{n}.$$

On the central path, each component satisfies $x_i z_i = \mu$, so the average relation holds exactly. Because our feasible-start method maintains primal and dual feasibility, μ reflects only the remaining *duality gap* between primal and dual objectives. Since the iterates are not exactly on the central path, the next target value is set as

$$\mu_{\text{target}} = \sigma\,\frac{x^\top z}{n}, \qquad \sigma \in (0, 1).$$

The parameter σ controls the centering: values close to 1 give safer but slower progress, while smaller values produce more aggressive steps that close the duality gap faster. In our implementation we take $\sigma = 0.1$.

Strengths and Limitations Primal–dual interior-point methods have several fundamental strengths:

- they maintain primal and dual feasibility (or at least remain very close to it) at each iteration;
- they provide accurate control of the duality gap through the relaxed complementarity condition ($\lambda_i s_i = \mu$ in the generic inequality form; in our model with $x \geq 0$, the relaxed complementarity condition reads $x_i z_i = \mu$);
- they achieve polynomial-time convergence, with iteration complexity bounded by $O\big(\sqrt{\nu}\,\log(1/\varepsilon)\big)$;
- they are broadly applicable across LP, QP, SOCP, and SDP problems.

The main drawback is the per-iteration cost of solving the Newton linear systems, which in general requires expensive matrix factorizations. In practice, however, this cost can be substantially reduced by exploiting problem structure. In our case, the block-diagonal structure of E allows the Schur complement $S = EH^{-1}E^\top$ to be assembled and factorized efficiently, avoiding the formation of the full $(n+|K|)\times(n+|K|)$ system matrix. Combined with the excellent numerical stability of the method and the relatively small number of iterations usually required, this makes the approach highly competitive.

4 Practical Relevance and Applications

An additional motivation for considering interior-point methods lies in their proven effectiveness in large-scale applications. A notable example is provided by Ferris and Munson (2003), who demonstrated the efficiency of primal–dual interior-point algorithms for the training of *Support Vector Machines (SVMs)*, one of the most widely used tools in machine learning. The underlying optimization problem is a quadratic program with relatively few linear constraints and a Hessian matrix Q that can be interpreted as a low-rank update of a positive semidefinite matrix.

In this setting, interior-point methods have proven highly competitive. By exploiting problem structure and employing advanced numerical techniques (such as sparse factorization or the Sherman–Morrison–Woodbury formula), they achieved performance significantly superior to classical alternatives. Reported results include speedups by factors of several hundred and the ability to solve problems with tens of millions of variables through out-of-core implementations that handle datasets larger than memory.

Such examples strengthen the rationale for investigating the *Primal–Dual Feasible Interior-Point Method* in this Project. If these methods can successfully address massive SVM training problems, it is reasonable to expect that they may also be particularly well suited for our constrained quadratic program on a simplicial domain.

5 Pseudocode

The previous sections derived the feasible-start primal–dual interior-point framework for our quadratic program, highlighting how primal and dual feasibility are preserved while complementarity is progressively reduced. Having established the theoretical formulation and the simplified Newton system, we can now translate these steps into an explicit computational procedure.

From theory to implementation. Among the possible algorithmic variants, the two main choices are the *feasible-start* and the *infeasible-start* methods. The latter allows initialization from arbitrary (possibly infeasible) points, but in our case the feasible region is a compact polyhedron, a Cartesian product of simplices, so a strictly feasible starting point can be obtained without special preprocessing; for instance by assigning uniform positive values within each simplex block. Hence, adopting the feasible-start variant provides conceptual clarity and numerical stability.

In the exact feasible-start formulation, primal and dual feasibility are preserved at every iteration, so $r_P = r_D = 0$ analytically. In implementation, however, finite-precision effects may introduce small residuals, which are monitored through $\|r_P\|_\infty$, $\|r_D\|_\infty$, and μ in the stopping criterion.

Algorithm 1: Feasible-Start Primal–Dual Interior-Point Method (Fixed Centering Parameter)

Input: $Q \succeq 0$, q, block-equality matrix E, tolerances ϵ_{feas}, ϵ_{comp}, centering parameter $\sigma \in (0, 0.5)$, maximum iterations *max_iter*

Output: Approximate optimizer (x, y, z) satisfying $Ex = \mathbf{1}$, $x > 0$, $z > 0$, $Qx + q + E^\top y - z \approx 0$

Step 1: Initialization.;

For each block I^k: set $x_i \leftarrow 1/|I^k|$ for $i \in I^k$; `//` $Ex = \mathbf{1}$, $x > 0$

$w \leftarrow Qx + q$; choose $\delta_k > 0$ and set $y_k \leftarrow -\min_{i \in I^k} w_i + \delta_k$ for all k;;

$z \leftarrow w + E^\top y$; `//` $z > 0$ `by construction`

$\mu \leftarrow (x^\top z)/n$.;

Step 2: Iterative loop.;

for $t = 1$ **to** *max_iter* **do**

- **Step 2.1: Residual computation.**;
- $r_D \leftarrow Qx + q + E^\top y - z$; $r_P \leftarrow Ex - \mathbf{1}$.;
- **Step 2.2: System assembly.**;
- $X \leftarrow \operatorname{diag}(x)$, $Z \leftarrow \operatorname{diag}(z)$, $M \leftarrow Z + XQ$; $\tilde{\mu} \leftarrow \sigma\,\mu$; $r_C \leftarrow XZ\mathbf{1} - \tilde{\mu}\mathbf{1}$.;
- $S \leftarrow EM^{-1}XE^\top$; $b \leftarrow -EM^{-1}r_C$.;
- **Step 2.3: Direction computation.**;
- Solve $S\,\Delta y = b$; $\Delta x \leftarrow M^{-1}(-r_C - XE^\top \Delta y)$; $\Delta z \leftarrow Q\,\Delta x + E^\top \Delta y$.;
- **Step 2.4: Step-size selection.**;
- $\alpha_{\text{pri}} \leftarrow \min\Big(1,\ 0.99 \cdot \min_{i:\,\Delta x_i < 0} -x_i/\Delta x_i\Big)$;;
- $\alpha_{\text{dual}} \leftarrow \min\Big(1,\ 0.99 \cdot \min_{i:\,\Delta z_i < 0} -z_i/\Delta z_i\Big)$;;
- $\alpha \leftarrow \min(\alpha_{\text{pri}}, \alpha_{\text{dual}})$.;
- **Step 2.5: Update.**;
- $x \leftarrow x + \alpha\,\Delta x$; $y \leftarrow y + \alpha\,\Delta y$; $z \leftarrow z + \alpha\,\Delta z$; $\mu \leftarrow (x^\top z)/n$.;
- **Step 2.6: Stopping criterion.**;
- **if** $\|r_P\|_\infty \le \epsilon_{\text{feas}}$ ***and*** $\|r_D\|_\infty \le \epsilon_{\text{feas}}$ ***and*** $\mu \le \epsilon_{\text{comp}}$ **then**
  - **break**

6 From Pseudocode to Python

Algorithm 1 describes the feasible-start primal–dual IPM in idealized form: it assumes exact arithmetic and exact primal–dual feasibility at every iterate. In finite precision, however, regularization, inexact linear solves, and step lengths $\alpha < 1$ introduce small deviations from exact feasibility. For this reason, the implementation in `solver.py` (class `IPM`) adopts a numerically robust variant, differing from Algorithm 1 in four main respects:

1. **Symmetric H formulation.** The Newton system is solved using $H := Q + X^{-1}Z$ rather than $M = Z + XQ$. Although both forms are mathematically equivalent (via $M = XH$), H is symmetric positive definite and admits a stable Cholesky factorization, whereas M is not symmetric and cannot be safely factorized by Cholesky.

2. **Active residual correction.** The full residuals r_P and r_D are included in the Newton right-hand sides at every iteration, actively correcting feasibility drift rather than only monitoring it in the stopping criterion.

3. **Corrected dual update.** The exact formula $\Delta z = Q\Delta x + E^\top \Delta y + r_D$ is used in place of $\Delta z = Q\Delta x + E^\top \Delta y$, which is only valid when $r_D = 0$.

4. **Regularisation and safeguards.** A small fixed diagonal shift $\tau_{\text{reg}} I$ (with $\tau_{\text{reg}} = 10^{-8}$) is added to H to improve conditioning, and the components x_i are clamped below by 10^{-14} before forming $X^{-1}Z = \operatorname{diag}(z_i/x_i)$, in order to avoid numerical instability in the division near the boundary.

6.1 The H Formulation: Derivation and Correctness

Eliminating Δz from the Newton system yields the reduced system in $(\Delta x, \Delta y)$:

$$H\,\Delta x + E^\top \Delta y = \text{rhs}_x, \qquad E\,\Delta x = \text{rhs}_y,$$

where the coefficient matrix of Δx is

$$H := Q + X^{-1}Z, \qquad X^{-1}Z = \operatorname{diag}(z_i/x_i).$$

Since $Q \succeq 0$ and $z_i/x_i > 0$ for all i on the interior, H is symmetric positive definite:

$$v^\top H v = v^\top Q v + \sum_i \frac{z_i}{x_i}\, v_i^2 > 0 \qquad \forall\, v \neq 0.$$

The Schur complement $S := EH^{-1}E^\top$ inherits positive definiteness from H (since the rows of E are indicator vectors of disjoint blocks, E has full row rank), so $S\,\Delta y = b$ has a unique solution. The full Newton directions follow by back-substitution:

$$\Delta x = H^{-1}\big(-X^{-1}r_C - r_D - E^\top \Delta y\big), \qquad \Delta z = Q\,\Delta x + E^\top \Delta y + r_D.$$

6.2 Implementation Structure

The solver is implemented as a single Python class `IPM` in `solver.py`. It accepts Q (dense or sparse), q, and the block partition, then builds the constraint matrix $E \in \mathbb{R}^{|K|\times n}$ as a sparse `scipy.sparse.csr_matrix`. This representation allows all products Ex and $E^\top y$ to be computed as standard sparse matrix–vector multiplications.

Initialization. The solver constructs a strictly feasible starting point (x^0, y^0, z^0) satisfying $Ex^0 = \mathbf{1}$, $x^0 > 0$, $z^0 > 0$, and $Qx^0 + q + E^\top y^0 - z^0 = 0$. Setting $x_i^0 = 1/|I_k|$ for $i \in I_k$ and $w := Qx^0 + q$, we choose for each block k:

$$y_k^0 := -\min_{i\in I_k} w_i + \delta_k, \qquad z_i^0 := w_i + y_k^0 \geq \delta_k > 0,$$

with $\delta_k = \max\{\varepsilon,\ \tau_\delta \operatorname{range}(w_{I_k})\}$, $\varepsilon = 10^{-8}$, $\tau_\delta = 10^{-2}$.

6.3 Implementation Notes

We now summarize the main implementation choices adopted in the solver.

1. Sparse constraint matrix E. The block-summation matrix $E \in \mathbb{R}^{|K| \times n}$ is built once in the constructor as a `scipy.sparse.csr_matrix`. Its action is

$$(Ex)_k = \sum_{i \in I^k} x_i, \qquad k = 1, \ldots, |K|,$$
$$(E^\top y)_i = y_k, \qquad i \in I^k,$$

and is evaluated via standard sparse matrix–vector products `E @ x` and `E.T @ y`, both in $O(n)$ time.

2. Symmetric H and Cholesky factorization. The solver works with the symmetric system $H = Q + X^{-1}Z + \tau_{\text{reg}} I$, which is factorized at each iteration. For dense Q, we use Cholesky factorization (`cho_factor`/`cho_solve`); for sparse Q, we use `splu` from SciPy sparse.

3. Fixed regularisation of H. A diagonal perturbation $H \leftarrow H + \tau_{\text{reg}} I$ is applied with $\tau_{\text{reg}} = 10^{-8}$ (fixed). This is sufficient for all tested problem instances and avoids the complexity of adaptive regularisation strategies.

4. Inclusion of r_P and r_D in the Newton system. To control numerical drift, the implementation always includes the primal and dual residuals r_P and r_D in the Newton system, rather than treating them only as stopping diagnostics. This yields the Schur right-hand side

$$b := r_P - E\, H^{-1}(r_D + X^{-1} r_C),$$

and the back-substitution formulas

$$\Delta x = H^{-1}\big(-X^{-1} r_C - r_D - E^\top \Delta y\big), \qquad \Delta z = Q\, \Delta x + E^\top \Delta y + r_D.$$

5. Schur complement assembly and solve. The Schur complement $S = EH^{-1}E^\top \in \mathbb{R}^{|K| \times |K|}$ is always assembled explicitly, column by column: for each standard basis vector $e_j \in \mathbb{R}^{|K|}$, compute $S_{:,j} = E\, H^{-1}(E^\top e_j)$. Each column reuses the already-computed factorization of H. The resulting dense $|K| \times |K|$ system $S\, \Delta y = b$ is then solved by Cholesky factorization, exploiting the positive definiteness of S.

6. Centering parameter σ and complementarity target. At each iteration the complementarity target is $\mu_t = \sigma \mu$ with a fixed $\sigma = 0.1$, and the complementarity residual is $r_C = x \circ z - \mu_t\, \mathbf{1}$.

7. Fraction-to-boundary step sizing. Step lengths are chosen with the standard fraction-to-the-boundary rule, using safety factor $\gamma = 0.99$, and the actual update uses $\alpha = \min(\alpha_{\text{pri}}, \alpha_{\text{dual}})$.

7 Experimental Setup and Results

In order to test both the correctness and the computational performance of our solver, we conducted a systematic set of experiments comparing our feasible-start primal–dual interior-point implementation against a standard convex QP solver (CVXPY) used as a reference baseline. The comparison is structured along three complementary axes. First, we vary the *problem size* n, ranging from small instances ($n = 50$) up to large-scale ones ($n = 10{,}000$), to assess how both solvers scale with the number of variables. Second, we vary the *number of simplex blocks* $|K|$, keeping n fixed, to isolate the effect of the constraint structure on convergence and runtime. Third, we vary the *sparsity* of the Hessian Q, testing density levels from 100% down to 0.1%, to evaluate robustness and efficiency on sparse instances. Across all configurations, we record wall-clock solve time, convergence status, and the relative error between the two objective values, providing a comprehensive picture of when the solver is competitive and where further optimization effort would be most beneficial.

7.1 Setup

Problem generation. Problems of different sizes are generated via `create_example_problem()`: $Q = A^\top A + \varepsilon I$ with $\varepsilon = 0.1$ (ensures $Q \succ 0$); for dense problems A has i.i.d. standard Gaussian entries, for sparse problems A is a random sparse matrix with entries uniform in $[0, 1)$. The linear cost q has i.i.d. standard Gaussian entries. Blocks have equal size $n/|K|$. All experiments use `seed=42` for reproducibility.

Solvers compared.

1. **Baseline (CVXPY)**: convex QP solved by CVXPY (reference).
2. **IPM**: our feasible-start primal–dual IPM (`IPM`).

Metrics. Wall-clock solve time (seconds), convergence flag, and objective agreement measured as objective relative error (as printed by the benchmark suite). For the largest instance we also report the absolute objective difference and the solution discrepancy $\|x - x_{\text{baseline}}\|_\infty$.

Reproducibility (CLI). All experiments were executed via the thin CLI entrypoint `benchmark_runner.py`, which calls `run_suite` / `run_benchmark` in `simplex_ipm.helper.benchmark`. The full suite is reproducible with:

```
uv run benchmark_runner.py --suite --seed 42
```

and individual instances can be run, e.g.,

```
uv run benchmark_runner.py --n 10000 --n-blocks 200 --density 0.01 --seed 42.
```

7.2 Suite summary

Table 1 reports the benchmark suite summary (18 configurations, seed = 42). The **Ratio** column follows the log output convention: it reports a factor ≥ 1 and uses $\uparrow$ if IPM is faster than CVXPY, $\downarrow$ otherwise.

Table 1: Benchmark suite summary: CVXPY vs IPM, seed = 42.

Problem	n	$\|K\|$	dens	CVXPY (s)	IPM (s)	Ratio	Obj rel err	Conv
`scale_n50`	50	5	1.00	0.0120	0.0053	2.27× ↑	1.09×10^{-8}	✓
`scale_n100`	100	10	1.00	0.0135	0.0108	1.25× ↑	3.48×10^{-8}	✓
`scale_n500`	500	50	1.00	0.1316	0.3181	2.42× ↓	1.39×10^{-9}	✓
`scale_n1000`	1000	100	1.00	0.6475	1.5357	2.37× ↓	3.63×10^{-10}	✓
`scale_n2000`	2000	200	1.00	3.8021	17.0183	4.48× ↓	3.41×10^{-10}	✓
`sparse_100pct`	1000	50	1.00	0.5307	1.1154	2.10× ↓	4.38×10^{-9}	✓
`sparse_50pct`	1000	50	0.50	0.6244	1.2577	2.01× ↓	2.85×10^{-11}	✓
`sparse_10pct`	1000	50	0.10	0.5884	1.3664	2.32× ↓	6.55×10^{-10}	✓
`sparse_1pct`	1000	50	0.01	0.3356	1.1761	3.50× ↓	8.21×10^{-8}	✓
`blocks_k2`	500	2	1.00	0.1126	0.1190	1.06× ↓	7.87×10^{-7}	✓
`blocks_k10`	500	10	1.00	0.1129	0.1418	1.26× ↓	3.00×10^{-8}	✓
`blocks_k50`	500	50	1.00	0.1321	0.3194	2.42× ↓	1.39×10^{-9}	✓
`blocks_k100`	500	100	1.00	0.1937	0.4734	2.44× ↓	1.71×10^{-10}	✓
`blocks_k250`	500	250	1.00	0.2590	0.9535	3.68× ↓	1.35×10^{-11}	✓
`lg_n5000_d1pct`	5000	100	0.01	50.7681	124.2526	2.45× ↓	6.10×10^{-8}	✓
`lg_n5000_d0.1pct`	5000	100	0.001	13.4041	128.4958	9.59× ↓	1.56×10^{-7}	✓
`lg_n10000_d1pct`	10000	200	0.01	384.8964	959.2357	2.49× ↓	5.55×10^{-9}	✓
`lg_n10000_d0.1pct`	10000	200	0.001	249.4369	1426.1233	5.72× ↓	1.15×10^{-7}	✓

Ratio convention: ↑ = IPM faster than CVXPY, ↓ = IPM slower.

Takeaway. Across all 18 configurations, the IPM converged and matched the CVXPY objective closely (objective relative errors between 10^{-11} and 10^{-7}). In this suite, CVXPY is faster on medium and large instances, while the IPM is faster on the two smallest dense problems (`scale_n50`, 2.27×; `scale_n100`, 1.25×).

7.3 Detailed single-instance comparison

Table 2 reports the detailed comparison for the largest instance: $n = 10{,}000$, $|K| = 200$, density = 0.01 (suite entry `lg_n10000_d1pct`).

Table 2: Single-instance benchmark ($n = 10{,}000$, $|K| = 200$, density = 0.01, seed = 42).

Solver	**Time (s)**	**Iter**	**Objective**	**Status**	**Feasible**	**Speed**
Baseline (CVXPY)	384.8964	–	3.5596411264×10^3	Converged	✓	1.00×
IPM	959.2357	18	3.5596411462×10^3	Converged	✓	2.49× slower

Accuracy diagnostics	**Value**
Objective difference Δf (IPM − CVXPY)	1.975847×10^{-5}
Solution difference $\|x - x_{\text{CVXPY}}\|_\infty$	1.194387×10^{-4}

8 Conclusions

We implemented a feasible-start primal–dual interior-point method for convex quadratic programming on a Cartesian product of simplices, and benchmarked it against a CVXPY baseline on 18 configurations (seed = 42), ranging from $n = 50$ to $n = 10{,}000$.

Correctness. The IPM converged on all 18 tested instances and matched the CVXPY reference solution with high accuracy, with objective relative errors between 10^{-11} and 10^{-7}. On the largest tested instance ($n = 10{,}000$, $|K| = 200$, density $= 0.01$) the objective difference was $\Delta f \approx 1.98 \times 10^{-5}$, with solution discrepancy $\|x - x_{\mathrm{CVXPY}}\|_\infty \approx 1.19 \times 10^{-4}$. These results confirm that the feasible-start initialization, the symmetric $H = Q + X^{-1}Z$ formulation, and the fraction-to-boundary step rule together provide a robust and numerically stable implementation.

Runtime. In this benchmark suite, CVXPY is faster on medium and large instances. At $n = 10{,}000$, $|K| = 200$, density $= 0.01$, the IPM required 959.24 seconds versus 384.90 seconds for CVXPY (2.49× slower), converging in 18 IPM iterations. The small iteration count is consistent with the theoretical bound $O(\sqrt{\nu}\log(1/\varepsilon))$, confirming that the algorithm follows the central path efficiently; the bottleneck lies instead in the per-iteration cost of assembling and solving the Schur system in Python. Two additional structural effects emerge from the suite: (i) the performance gap widens as $|K|$ grows (from 1.06× at $|K| = 2$ to 3.68× at $|K| = 250$), since a larger number of blocks increases the size of the Schur complement system; (ii) sparsity in Q does not bring substantial relief to our solver in the tested range, as the dominant cost remains in the Schur solve rather than in matrix–vector products with Q. The IPM is faster on the two smallest dense problems (`scale_n50`, 2.27×; `scale_n100`, 1.25×).

Interpretation and outlook. It should be emphasized that CVXPY dispatches the problem to highly optimized C-level solvers (e.g., operator-splitting or interior-point implementations written in compiled languages), whereas our solver is a pure-Python implementation. The observed runtime gap should therefore not be interpreted as a structural weakness of the interior-point methodology itself.

From a theoretical and algorithmic standpoint, the method remains highly competitive: across all experiments, convergence was achieved in a small number of IPM iterations (12–19 across the suite), in line with the expected $O(\sqrt{\nu}\log(1/\varepsilon))$ complexity bound. This confirms that the iterates closely follow the central path and that the centering and step-length strategies are effective.

The dominant computational bottleneck is purely linear-algebraic. Each iteration requires the factorization (or iterative solution) of the Schur complement system, whose cost grows with both n and $|K|$. In the current implementation, this step is executed in Python with high-level linear algebra routines, which limits scalability. Consequently, the present performance gap should be regarded as an engineering challenge, related to low-level optimization, sparse factorization strategies, and possible compiled backends, rather than as a fundamental limitation of primal–dual interior-point methods for this class of structured quadratic programs.

A Configuration Parameters

Parameter	Default	Description
`sigma`	0.1	Centering $\sigma \in (0, 0.5)$
`max_iter`	100	Maximum IPM iterations
`eps_feas`	10^{-8}	Feasibility tolerance
`eps_comp`	10^{-8}	Complementarity tolerance
`eps_delta`	10^{-8}	Minimum δ_k
`tau_delta`	10^{-2}	Scaling factor for δ_k
`tau_reg`	10^{-8}	Fixed regularization τ
`gamma`	0.99	Fraction-to-boundary factor
`verbosity`	1	0=silent, 1=summary, 2=per-iteration